



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

دانشکده‌ی مهندسی کامپیوتر

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین اول

فهرست مسائل

۱	 مسئله ۱
۱	 الف) چسبندگی (Cohesion)
۱	 ب) اصل LSP در SOLID
۲	 مسئله ۲
۳	 مسئله ۳
۳	 ۱. آیا پروژه شما معماری دارد؟ چرا؟
۳	 ۲. آیا پروژه‌هایی وجود دارد که معماری نداشته باشند؟
۳	 ۳. در صورت وجود پروژه‌های بدون معماری، چه مشکلاتی دارند؟
۴	 مسئله ۴
۵	 مسئله ۵
۵	 ۱. چرا تمام ماژول‌های کاربردی اپ در ماژول app ادغام نشده‌اند؟
۵	 ۲. معماری این پروژه به کدام الگو نزدیک‌تر است؟
۶	 ۳. چرا معماری نرم‌افزار به پروژه‌های وب محدود نمی‌شود؟
۷	 مسئله ۶
۷	 گزارش مراحل کار
۷	 بررسی صحت عملکرد برنامه
۹	 بررسی معماری پروژه و تحلیل کلی
۱۱	 قابلیت تغییرپذیری معماری برای اضافه کردن قابلیت جدید

پاسخ مسئله‌ی ۱.

الف) چسبندگی (Cohesion)

چسبندگی به میزان ارتباط و انسجام اجزای داخلی یک ماژول یا کلاس اشاره دارد. به عبارت دیگر، نشان می‌دهد که عناصر یک واحد نرم‌افزاری تا چه حد در راستای یک هدف مشخص و مشترک عمل می‌کنند.

در طراحی مناسب، هر کلاس یا ماژول باید یک مسئولیت مشخص و متمرکز داشته باشد. در این حالت، گفته می‌شود چسبندگی بالا است که یک ویژگی مطلوب محسوب می‌شود. در مقابل، اگر یک کلاس چندین وظیفه نامرتبط را بر عهده داشته باشد، چسبندگی آن پایین بوده و این موضوع باعث کاهش خوانایی، نگهداری‌پذیری و قابلیت توسعه سیستم می‌شود.

جمع‌بندی: چسبندگی بالا به معنای تمرکز هر واحد نرم‌افزاری بر یک هدف مشخص و واحد است.

ب) اصل LSP در SOLID

اصل جایگزینی لیسکوف بیان می‌کند که اشیاء یک کلاس پایه باید بتوانند بدون ایجاد اختلال در درستی برنامه، با اشیاء کلاس‌های مشتق‌شده جایگزین شوند.

به بیان دقیق‌تر، کلاس‌های فرزند باید رفتارهایی سازگار با قرارداد تعریف‌شده در کلاس پایه ارائه دهند و نباید پیش‌شرط‌ها را سخت‌تر یا پس‌شرط‌ها را ضعیف‌تر کنند. در غیر این صورت، استفاده از Polymorphism منجر به بروز خطاهای منطقی در سیستم خواهد شد.

مثال معروف: فرض کنید کلاس Rectangle را داریم. اگر بخوایم کلاس Square را بسازیم و از Rectangle ارث ببرد، به مشکل می‌خوریم! چون اگر عرض را تغییر دهیم، طول هم باید تغییر کند. اما Rectangle انتظار دارد که ما می‌توانیم عرض و طول را جداگانه عوض کنیم. پس اگر جایی از کد Rectangle استفاده می‌کنیم و ما به جای آن Square بزاریم، منطق و کد به مشکل می‌خورد.

پاسخ مسئله‌ی ۲.

خیر، لزوماً این‌طور نیست.

متخصص شدن در یک حوزه‌ی خاص از مهندسی نرم‌افزار (مثلاً فقط پایگاه‌داده، فرانت‌اند یا الگوریتم‌ها) به این معنا نیست که آن فرد حتماً معمار نرم‌افزار خوبی هم خواهد شد. معماری نرم‌افزار نیازمند دیدی گسترده‌تر از صرفاً تسلط بر یک موضوع است.

یک معمار خوب باید بتواند بین بخش‌های مختلف سیستم ارتباط برقرار کند، تصمیم‌های کلان بگیرد، و trade-off ها (مثل کارایی، مقیاس‌پذیری، نگهداری‌پذیری و امنیت) را به‌درستی مدیریت کند. این مهارت‌ها معمولاً از تجربه در حوزه‌های مختلف، درک کلی از سیستم‌ها، و توانایی تحلیل مسائل در سطح بالا به دست می‌آیند، نه فقط از عمیق شدن در یک زمینه‌ی خاص. به طور کلی بهتر است حوزه‌های تخصصی معمار نرم‌افزار T-Shaped باشد. (رفرنس به درس ASD دکتر رامسین)

بنابراین، تخصص عمیق در یک حوزه می‌تواند یک مزیت باشد، اما برای تبدیل شدن به یک معمار خوب کافی نیست؛ بلکه باید در کنار آن، دید سیستمی و تجربه‌ی گسترده نیز وجود داشته باشد.

پاسخ مسئله‌ی ۳.

۱. آیا پروژه شما معماری دارد؟ چرا؟

بله، تقریباً هر پروژه‌ای حتی ساده‌ترین آن نوعی معماری دارد. معماری یعنی نحوه‌ی سازمان‌دهی اجزا، ارتباط بین بخش‌ها، و تصمیمات کلیدی طراحی. حتی اگر این تصمیمات به‌صورت آگاهانه گرفته نشده باشند، باز هم یک ساختار (مثلاً یک فایل ساده با توابع پشت سر هم) وجود دارد که می‌توان آن را نوعی معماری ابتدایی دانست.

۲. آیا پروژه‌هایی وجود دارد که معماری نداشته باشند؟

خیر.

۳. در صورت وجود پروژه‌های بدون معماری، چه مشکلاتی دارند؟

به‌طور دقیق، پروژه‌ای که هیچ معماری‌ای نداشته باشد وجود ندارد. هر سیستم نرم‌افزاری ناگزیر یک ساختار دارد، حتی اگر این ساختار بسیار ضعیف، بی‌نظم یا تصادفی باشد. داشتن معماری را نباید صرف داشتن ساختاری منظم و از پیش فکر شده در نظر گرفت. حتی یک اسکریپت یا یک فایل کوچک نیز معماری دارد.

پاسخ مسئله‌ی ۴.

۱. نحوه مدیریت وابستگی‌ها

• Layered Architecture :

- وابستگی‌ها به صورت بالا به پایین هستند (Presentation → Business → Data).
- هر لایه فقط لایه زیرین را می‌شناسد.
- جفت‌شدگی نسبتاً زیاد است.
- در صورت عدم استفاده از Dependency Injection، تغییرات اثر زنجیره‌ای دارند.

• MVVM :

- وابستگی‌ها از طریق ViewModel مدیریت می‌شوند.
- View به ViewModel وابسته است و Model مستقل می‌ماند.
- جفت‌شدگی کم (Loose Coupling).

۲. قابلیت تست‌پذیری

• Layered Architecture :

- تست‌پذیری مناسب در لایه Business. البته در صورتی که DI به درستی استفاده شده باشد.
- تست لایه Presentation دشوارتر است.
- نیاز به Mock کردن لایه‌ها.

• MVVM :

- تست‌پذیری بسیار بالا.
- امکان تست ViewModel بدون UI.

۳. میزان پیچیدگی پیاده‌سازی

• Layered Architecture :

- ساده و قابل فهم.
- مناسب پروژه‌های کوچک تا متوسط.
- پیاده‌سازی سریع.

• MVVM :

- پیچیده‌تر.
- نیازمند مفاهیمی مانند Data Binding و Command Pattern.
- دارای سربار اولیه بیشتر.

۴. انعطاف‌پذیری در تغییرات رابط کاربری

• Layered Architecture :

- تغییر UI ممکن است روی سایر لایه‌ها اثر بگذارد.
- انعطاف‌پذیری کمتر.

• MVVM :

- انعطاف‌پذیری بالا.
- امکان تغییر UI بدون تغییر در Model و ViewModel.
- مناسب برای رابط‌های کاربری پویا.

پاسخ مسئله‌ی ۵.

پروژه Now in Android به صورت ماژولار سازمان‌دهی شده و ماژول‌ها معمولاً در سه دسته‌ی اصلی قرار می‌گیرند: app، core و feature. ماژول app نقش نقطه‌ی ورود را دارد؛ یعنی راه‌اندازی اپ، پیکربندی DI، مسیریابی/ناوبری، اتصال صفحه‌ها به یکدیگر و تعریف وابستگی‌ها در سطح برنامه عمده‌تاً در این ماژول انجام می‌شود و خود منطق قابلیت‌ها تا حد امکان در آن قرار نمی‌گیرد. ماژول‌های core بخش‌های زیرساختی و مشترک را فراهم می‌کنند (مانند core:designsystem و core:model، core:datastore، core:database، core:network) و به عنوان سرویس‌های عمومی در اختیار سایر بخش‌ها قرار می‌گیرند تا قابلیت استفاده مجدد و کپسوله‌سازی حفظ شود. در مقابل، ماژول‌های feature هر کدام یک قابلیت/صفحه‌ی مستقل از اپ (مثل علاقه‌مندی‌ها، بوکمارک‌ها، جستجو و ...) را شامل می‌شوند و معمولاً UI و منطق ارائه‌ی همان دامنه را نگه می‌دارند و برای دریافت داده به core (یا core:data) وابسته‌اند؛ این تفکیک باعث می‌شود توسعه، تست، و نگهداری هر قابلیت مستقل‌تر و مقیاس پذیرتر انجام شود.

۱. چرا تمام ماژول‌های کاربردی اپ در ماژول app ادغام نشده‌اند؟

ادغام کردن تمام کدها در یک ماژول app در پروژه‌های کوچک ممکن است قابل قبول باشد، اما در پروژه‌های واقعی و بزرگ باعث افزایش پیچیدگی و هزینه نگهداری می‌شود. دلایل اصلی ماژولار بودن در پروژه NiA عبارت‌اند از:

- **افزایش سرعت ساخت و کامپایل:** با جدا کردن قابلیت‌ها در ماژول‌های مستقل، Gradle می‌تواند فقط ماژول‌های تغییر یافته را بازسازی کند و حتی بخشی از کارها را موازی‌سازی کند. این موضوع در پروژه‌های بزرگ بسیار تاثیرگذار است.
- **Separation of Concerns:** هر ماژول مسئول یک دامنه مشخص است مثلاً feature:search یا feature:bookmarks و این باعث می‌شود کدها کمتر به هم وابسته شوند و از ایجاد Spaghetti Code جلوگیری شود.
- **کاهش وابستگی‌ها و کپسوله‌سازی:** ماژول‌ها می‌توانند API مشخصی ارائه دهند و جزئیات داخلی خود را مخفی نگه دارند. در نتیجه تغییرات داخلی یک ماژول، کمترین اثر را روی سایر بخش‌ها می‌گذارد.
- **توسعه تیمی و مقیاس‌پذیری:** در تیم‌های بزرگ، افراد/تیم‌های مختلف روی ویژگی‌های مختلف کار می‌کنند. تفکیک به ماژول‌ها احتمال Conflict و وابستگی‌های ناخواسته را کاهش می‌دهد.
- **Reusability:** ماژول‌های عمومی مانند core:network، core:database یا core:designsystem می‌توانند در پروژه‌های دیگر هم استفاده شوند بدون آنکه به منطق یا UI یک ویژگی خاص وابسته باشند.

۲. معماری این پروژه به کدام الگو نزدیک‌تر است؟

این پروژه در لایه Presentation بیشترین شباهت را به MVVM دارد، اما در سطح کلان‌تر (چیدمان ماژول‌ها و تفکیک لایه‌ها) می‌توان Clean Architecture را مشاهده کرد. به بیان دیگر، معماری آن یک ترکیب عملی از چند الگو/راهنمایی رایج در اندروید مدرن است.

• MVVM:

— **View:** رابط کاربری با Jetpack Compose پیاده‌سازی می‌شود و عمدتاً وظیفه آن نمایش داده‌ها و ارسال رویدادهای کاربر است (مانند کلیک، جستجو و ...).

— **ViewModel:** منطق نمایش و وضعیت صفحه در ViewModel نگهداری می‌شود. ViewModel داده‌های موردنیاز UI را از طریق جریان‌های واکنش‌گرا مانند StateFlow/Flow منتشر می‌کند و UI با مشاهده این‌ها به صورت خودکار Recompose می‌شود.

– **Model**: مدل‌ها و داده‌ها از طریق Repository ها و منابع داده (شبکه، دیتابیس، DataStore) تأمین می‌شوند و ViewModel مستقیماً درگیر جزئیات پیاده‌سازی شبکه/ذخیره‌سازی نمی‌شود. این موضوع باعث کاهش وابستگی‌ها و افزایش تست‌پذیری می‌گردد.

• **Repository Pattern و نزدیک به Clean Architecture (در ساختار کلی سیستم):**

– **UI Layer**: شامل ماژول‌های feature:* و ماژول‌های UI مشترک (مانند core:ui و core:designsystem) که نمایش، ناوبری و تعاملات کاربر را مدیریت می‌کنند.

– **Domain Layer**: در برخی بخش‌ها منطق تجاری با UseCase ها یا لایه‌های میانی مستقل از UI پیاده می‌شود تا قوانین کسب‌وکار از جزئیات نمایش جدا شوند.

– **Data Layer**: شامل ماژول‌هایی مانند core:data، core:network، core:database و core:datastore که مسئول دریافت، ذخیره و همگام‌سازی داده‌اند. در این لایه، Repository ها نقش واسط بین View-Model/Domain و منابع داده را بازی می‌کنند و تصمیم می‌گیرند داده از کجا (شبکه/کش/دیتابیس) خوانده شود.

در نتیجه، اگر بخواهیم یک جمع‌بندی دقیق ارائه کنیم: لایه Presentation پروژه عمدتاً MVVM است، و در سطح معماری کلان به Clean Architecture نزدیک شده.

۳. چرا معماری نرم‌افزار به پروژه‌های وب محدود نمی‌شود؟

معماری نرم‌افزار به وب محدود نیست، چون «معماری» به نوع پلتفرم وابسته نیست؛ بلکه روشی برای مدیریت پیچیدگی، کاهش هزینه تغییرات و افزایش کیفیت در هر سیستم نرم‌افزاری است. هر جا که یک سیستم شامل اجزای متعدد، وابستگی‌های گوناگون و نیاز به توسعه و نگهداری بلندمدت داشته باشد، معماری اهمیت پیدا می‌کند. دلایل اصلی عبارت‌اند از:

• همه سیستم‌های نرم‌افزاری با پیچیدگی و تغییرات مواجه‌اند: چه وب باشد، چه موبایل، چه دسکتاپ، چه بازی، چه سامانه‌های نهفته یا IoT، همگی با نیازمندی‌های جدید، تغییر فناوری، و رشد تدریجی سیستم روبه‌رو می‌شوند. معماری کمک می‌کند این تغییرات کنترل‌پذیر و کم‌ریسک‌تر شوند.

• **Non-Functional Requirements** عمومی‌اند: مفاهیمی مثل کارایی، مقیاس‌پذیری، امنیت، قابلیت اطمینان، نگهداری‌پذیری و تست‌پذیری فقط مخصوص وب نیستند و در همه نرم‌افزارها مطرح می‌شوند. معماری ابزار تصمیم‌گیری برای رسیدن به این ویژگی‌هاست.

• **تفکیک مسئولیت‌ها و مدیریت وابستگی‌ها** در همه‌جا لازم است: اصل Separation of Concerns و جلوگیری از وابستگی‌های درهم‌تنیده برای هر پروژه‌ای ضروری است؛ زیرا بدون آن، تغییر یک بخش باعث شکستن بخش‌های دیگر می‌شود و هزینه توسعه به‌مرور افزایش می‌یابد.

بنابراین معماری نرم‌افزار یک مفهوم مستقل از پلتفرم است و در هر پروژه‌ای که قرار است پایدار، قابل توسعه، قابل تست و قابل نگهداری باشد، نقش اساسی دارد.

پاسخ مسئله‌ی ۶.

گزارش مراحل کار

در توسعه پروژه Mini-NginX، مراحل کار به صورت زیر دنبال شد تا یک سرور HTTP سبک و قابل تنظیم پیاده‌سازی شود.

۱. **مرحله برنامه‌ریزی و طراحی:** ابتدا نیازهای پروژه تعریف شد، شامل پشتیبانی از تنظیمات YAML، انواع مسیرها (ریدایرکت، سرو فایل، پروکسی معکوس، سرو دایرکتوری) و استفاده از الگوی استراتژی برای هندلرها. معماری لایه‌ای انتخاب شد تا نگرانی‌ها جدا شوند.

۲. **پیاده‌سازی بارگذاری تنظیمات:** فایل config.go نوشته شد برای خواندن فایل YAML و تبدیل به ساختار داده‌ای. این شامل تعریف struct های Config و Route بود.

۳. **پیاده‌سازی سرور و مدیریت اتصالات:** در server.go، listener TCP روی پورت مشخص ایجاد شد و برای هر اتصال ورودی یک goroutine اختصاص یافت تا concurrency پشتیبانی شود.

۴. **پیاده‌سازی روتینگ و دیسپچ:** در router.go، پارسر خط درخواست اضافه شد، مسیرها با استفاده از پیشوند تطبیق داده شدند و با switch بر اساس نوع مسیر، دیسپچ به هندلر انجام شد.

۵. **پیاده‌سازی هندلرها:** چهار هندلر مستقل نوشته شد: Redirect، StaticFile، ReverseProxy ServeDir. هر کدام با استراتژی ساده پیاده‌سازی شدند.

۶. **افزودن ویژگی‌های کمکی:** لاگینگ به فایل و کنسول اضافه شد، و تست دستی با ابزارهایی مانند curl انجام شد.

بررسی صحت عملکرد برنامه

در ابتدا کانفیگ YAML به صورت زیر تعریف شد:

```

Assignments > Assignment01 > mini-nginx > config.yml
1  listen-on: 8080
2  paths:
3    "/static/":
4      type: servedir
5      target: "./static/"
6    "/go-to-google":
7      type: redirect
8      target: "https://google.com"
9    "/chat/":
10     type: reverseproxy
11     target: "localhost:3000"
12    "/file.pdf":
13     type: staticfile
14     target: "/Users/moein/Downloads/api-test-V2-1.pdf"
15    "/downloads/":
16     type: servedir
17     target: "/Users/moein/Downloads/"
18
19
20

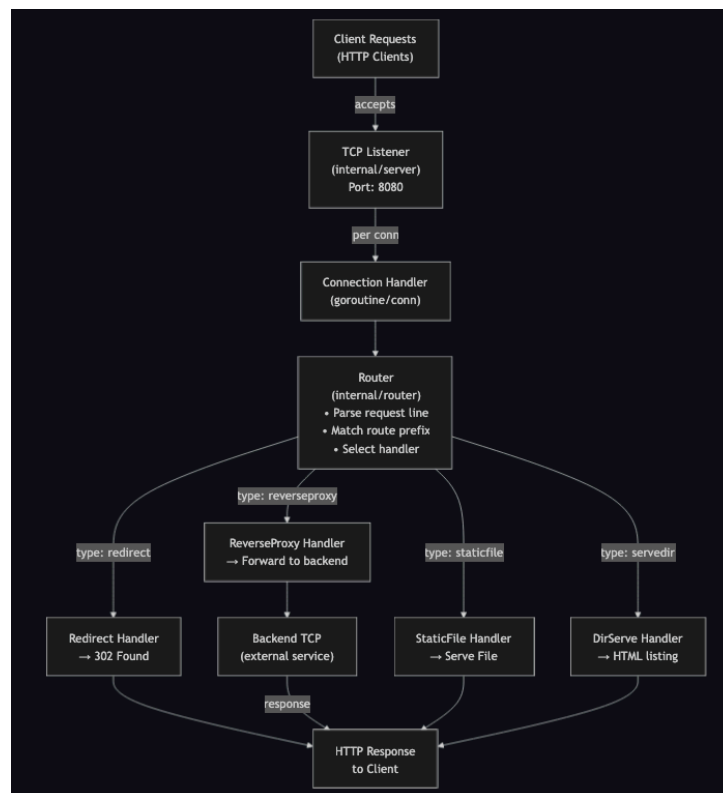
```

این کانفیگ از قابلیت‌های زیر پشتیبانی می‌کند:

- Serverdir
- StaticFile
- ReverseProxy

- Strategy Pattern: در لایه Application ، با استفاده از switch در router.go، رفتار هندلر بر اساس نوع مسیر انتخاب می‌شود. این پترن اجازه می‌دهد استراتژی‌های جدید (مانند هندلر جدید) بدون تغییر کد اصلی اضافه شوند. مثال: اضافه کردن case جدید برای "auth" نیاز به تغییر فقط در switch و پیاده‌سازی هندلر دارد.
- Configuration-as-Code: رفتارها در فایل YAML تعریف می‌شوند، نه hardcoded در کد.
- Dependency Injection: شیء Config به عنوان پارامتر به لایه‌های پایین‌تر پاس داده می‌شود تا وابستگی‌ها کاهش یابد.
- Concurrency Pattern: استفاده از goroutine per connection برای پشتیبانی از اتصالات همزمان.

معماری و فلووی کلی پروژه Mini-NginX به صورت زیر است:



ساختار فایل‌ها به این صورت است:

```

Mini-NginX/
├── cmd/
│   └── server/
│       └── main.go          # Entrypoint: load config, init logger, start server
├── internal/
│   ├── config/
│   │   └── config.go        # YAML schema + loader
│   ├── handlers/
│   │   ├── redirect.go      # 302 redirect responses
│   │   ├── reverseproxy.go  # Forwards request to backend and streams response
│   │   ├── servedir.go      # Directory serving/listing
│   │   └── staticfile.go     # Serves one file with detected MIME type
│   ├── logger/
│   │   └── logger.go         # Logger init and Info helper
│   ├── router/
│   │   └── router.go         # Request line parsing + route dispatch
│   └── server/
│       └── server.go         # TCP listener loop + connection handling
├── config.yml               # Route and port configuration
├── go.mod
├── go.sum
└── server.log                # Log output (created/overwritten at startup)
  
```

قابلیت تغییرپذیری معماری برای اضافه کردن قابلیت جدید

معماری لایه‌ای پروژه Mini-NginX بسیار تغییرپذیر است، به ویژه به خاطر استفاده از Strategy Pattern و جداسازی لایه‌ها.

برای اضافه کردن قابلیت جدید (مانند هندلر cache یا middleware)، کافی است:

۱. نوع جدید را به YAML اضافه کنید.

۲. case جدید در router.go switch اضافه کنید.

۳. هندلر جدید در handlers/ پیاده‌سازی کنید.

این کار بدون تغییر لایه‌های دیگر امکان‌پذیر است و اصل Open-Closed Principle رعایت می‌شود.